

Fazit und Ausblick

Ziel dieser Arbeit war es zu untersuchen, inwieweit sich ein KI-Agent zur automatisierten Generierung von CI/CD-Pipelines für CAP-Anwendungen realisieren lässt. Ausgangspunkt war die Beobachtung, dass das manuelle Erstellen solcher Pipeline-Konfigurationen zeitaufwändig und fehleranfällig ist und damit ein relevantes Automatisierungspotenzial besteht. Auf Basis des CRISP-DM-Prozesses wurde hierfür ein vollständiger Prototyp entwickelt, der CAP-Repositories analysiert und automatisch eine lauffähige Pipeline in Form einer `github/workflows/deploy.yml` bereitstellt.

Im Ergebnis zeigt sich, dass der Ansatz grundsätzlich funktioniert: Der Agent war in allen Testfällen in der Lage, ein valides Pipeline-Artefakt zu generieren. Die Qualität der Ergebnisse wurde im Rahmen einer Human-as-a-Judge-Evaluation beurteilt und anhand der Kriterien funktionale Vollständigkeit, syntaktische Korrektheit sowie praktische Effizienz bewertet. Besonders positiv fällt auf, dass nur vergleichsweise geringe Nacharbeiten nötig waren. Anpassungen erfolgten hauptsächlich in frühen Entwicklungsphasen (vor allem am System-Prompt), weniger an den generierten Pipelines selbst. Damit konnte gezeigt werden, dass ein generativer Ansatz die Erstellung von CI/CD-Konfigurationen spürbar vereinfachen und konsistenter gestalten kann.

Gleichzeitig wurde deutlich, dass der Erfolg maßgeblich von einer guten Datenbasis abhängt. Die getesteten CAP-Projekte wiesen teilweise unvollständige oder uneinheitliche Strukturen auf, die zunächst bereinigt oder ergänzt werden mussten. Insofern ist nicht allein der Agent ausschlaggebend, sondern ebenso der Reifegrad des jeweiligen Projekts. Darüber hinaus bildet die qualitative Evaluation valide erste Einschätzungen ab, ersetzt jedoch keine systematische quantitative Bewertung - insbesondere im Hinblick auf die Generalisierungsfähigkeit bei einer größeren Anzahl und Varianz von Projekttypen.

Ausblickend eröffnet der Ansatz interessante Möglichkeiten. Denkbar wäre etwa, den Agenten enger in die Entwicklungsumgebung zu integrieren, sodass Pipeline-Anpassungen automatisiert vorgeschlagen oder direkt validiert werden können. Auch eine Kombination mit anderen Agenten - etwa zur Codeanalyse, Security-Checks oder Projekt-Bootstrap-Generierung - erscheint vielversprechend. Parallel ließe sich der Fokus erweitern: Statt ausschließlich CAP-Projekte zu betrachten, könnten auch allgemeine Node.js- oder Multi-Service-Setups unterstützt werden.

Darüber hinaus existieren inzwischen alternative Werkzeuge, die in eine ähnliche Richtung gehen: GitHub-basiert etwa Copilot oder Cline, die kontextsensitiv Entwicklungsaufgaben unterstützen und Code-Artefakte erzeugen können; ebenso Cursor oder klassische LLMs wie ChatGPT oder Gemini. Im Gegensatz zu diesen generalisierten, oft rein textbasierten Assistenten wurde in dieser Arbeit ein hochspezialisierter Agent entwickelt. Der entscheidende Mehrwert dieses Ansatzes liegt in der Kombination aus Domänenspezialisierung und Steuerbarkeit: Der Agent ist durch den detaillierten System-Prompt und die Validierungsregeln exakt auf den Nischen-Anwendungsfall (SAP CAP auf Cloud Foundry) und dessen Best Practices (z.B. MTAR) abgestimmt. Er agiert als deterministisch gesteuerter Prozess (Analyse, Injektion, Validierung) und nicht als reiner Textgenerator. Diese Aspekte, kombiniert mit der sicheren Einbettung in die Unternehmenslandschaft (SAP AI Core), bieten Vorteile in puncto Sicherheit, Reproduzierbarkeit und fachlicher Korrektheit, die generische Tools nicht leisten können.

Insgesamt zeigt die Arbeit, dass ein KI-gestützter Agent zur Generierung von CI/CD-Pipelines für CAP-Anwendungen technisch umsetzbar ist und echten Mehrwert bietet. Zwar bleibt weiterer Entwicklungsaufwand notwendig - insbesondere in puncto Skalierbarkeit, einer quantitativen Evaluation auf einer breiteren Datenbasis sowie einer automatisierten Test-Validierung der generierten Artefakte - dennoch verdeutlichen die Ergebnisse das Potenzial, repetitive Entwicklungsaufgaben gezielt zu automatisieren und so Entwickler:innen zu entlasten. Damit kann

die Lösung ein Baustein sein, um die Software-Entwicklung effizienter und konsistenter zu gestalten, ohne den Menschen aus dem Entscheidungsprozess zu verdrängen.